МИНИСТЕРСТВО ОБРАЗОВАНИЯ КУЗБАССА

ГПОУ «ЮРГИНСКИЙ ТЕХНОЛОГИЧЕСКИЙ КОЛЛЕДЖ»
ИМ. ПАВЛЮЧКОВА Г.А
Отделение АВТОМАТИЗАЦИИ И ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

РАЗРАБОТКА КЛИЕНТСКОЙ ЧАСТИ ИНТЕРНЕТ-МАГАЗИНА МУЗЫКАЛЬНЫХ ИНСТРУМЕНТОВ

КУРСОВОЙ ПРОЕКТ
КП 09.02.09.454

Выполнил:
Студент группы № 454
_____ Молоков А.А.
(подпись) (ФИО студента)
«___» _____20__ г

Работа выполнена:
«___» _____ 20__г.
С оценкой «_____»

Руководитель _____ М.В. Поликарпочкин
(подпись) (ФИО руководителя)

2026

СОДЕРЖАНИЕ

Введение 3
1 Теоретические основы разработки интернет-магазина 5
1.1 Электронная коммерция и локальные бренды как объект автоматизации 5
1.2 Анализ существующих решений 6
1.3 Выбор технологического стека 7
1.4 Проектирование структуры данных 9
1.5 Выводы по первой главе 10
2 Практическая реализация интернет-магазина музыкальных инструментов13 12
2.1 Подготовка к разработке 12
2.2 Реализация главной страницы и главную страницу15 14
2.3. Разработка страницы каталога и визуализация товаров 18
2.4 Реализация страницы отдельного товара 21
2.5 Реализация корзины и оформления заказа 23
2.6 Подготовка краткого руководства оператора 27
2.7 Расширение работоспособности системы .. **Ошибка! Закладка не определена.**
2.8 Выводы по второй главе 27
Заключение 29
Список использованных источников 31
Приложение А - Связи сущностей 32
Приложение Б - Оформление страниц 33

ВВЕДЕНИЕ

В современных условиях развития электронной коммерции и роста популярности локальных брендов малый бизнес сталкивается с необходимостью выхода в онлайн. Специализированные музыкальные магазины, продающие музыкальные инструменты и звуковое оборудование, все чаще нуждаются в собственных интернет-магазинах для прямых продаж, минуя перекупщиков. Однако стандартные шаблонные решения зачастую не позволяют передать уникальную атмосферу бренда и привлечь современную аудиторию. Для повышения конкурентоспособности и улучшения пользовательского опыта (UX) требуется индивидуальный подход к разработке интерфейса, включающий интерактивные элементы, качественную визуализацию и продуманную архитектуру. Использование современных веб-технологий, таких как HTML5, CSS3 и JavaScript, представляет собой оптимальное решение для создания быстрого, безопасного и визуально привлекательного интернет-магазина, способного автоматизировать процесс продаж.

Объектом исследования является деятельность музыкального магазина по реализации музыкальных инструментов и процесс взаимодействия с клиентами при оформлении заказов через интернет.

Предметом исследования являются методы и средства разработки клиентской части интернет-магазина, а также инструменты создания интерактивного пользовательского интерфейса.

Целью курсовой работы является разработка клиентской части интернет-магазина музыкальных инструментов с уникальным интерактивным дизайном и автоматизацией процесса оформления заказов.

Задачами исследования для достижения поставленной цели являются:

1) Изучить предметную область, выделить целевую аудиторию и основные сценарии покупки музыкальных инструментов.

2) Проанализировать существующие аналоги (интернет-магазины музыкальных инструментов) по критериям UX/UI и функционала.

3) Спроектировать структуру интернет-магазина и разработать макеты основных страниц (Главная, Каталог, Страница товара, Корзина).

4) Реализовать клиентскую часть приложения с использованием технологий HTML5, CSS3 (Flexbox, Grid, анимации) и JavaScript для создания динамических элементов (фильтрация, корзина, анимации).

5) Реализовать интерактивные 3D-карточки товаров с эффектами наведения.

6) Реализовать локальную модель хранения данных на основе JSON для хранения товаров и обработки поступающих заказов.

7) Провести тестирование разработанной системы, проверить ее работоспособность и адаптивность под мобильные устройства.

Методами исследования послужили системный анализ предметной области, сравнительный анализ программных продуктов, UX/UI-проектирование, а также практическое применение технологий веб-разработки (клиентской части).

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ИНТЕРНЕТ-МАГАЗИНА

1.1 Электронная коммерция и локальные бренды как объект автоматизации

В современном мире локальные производители и специализированные магазины музыкальных инструментов, всё острее нуждаются в прямом канале сбыта своей продукции. Традиционная модель продаж через посредников и перекупщиков существенно снижает маржинальность бизнеса и не позволяет конечному потребителю узнать историю продукта. Создание собственного интернет-магазина решает эту проблему, позволяя выстроить прямую коммуникацию с клиентом.

Однако стандартные шаблонные решения (такие как готовые темы на базе WordPress, конструкторы Tilda или Wix) зачастую выглядят безлико и не способны передать уникальную атмосферу конкретного бренда. Кроме того, они часто перегружены лишним кодом, что снижает скорость загрузки. Для привлечения современной аудитории, привыкшей к качественному цифровому контенту, требуется уникальный UX/UI-дизайн, элементы геймификации и высокая производительность.

1.2 Анализ существующих решений

В ходе предпроектного исследования был проведен анализ популярных интернет-магазинов, специализирующихся на продаже музыкальных инструментов. В качестве прямых конкурентов были рассмотрены сайты: «Музторг» (muztorg.ru), «Pop-Music» (pop-music.ru) и «МедиаМаркт» (mediamarkt.ru).

1.2.1 Сайт «Музторг» (muztorg.ru)Данный ресурс представляет собой классический, хорошо проработанный интернет-магазин с упором на доверие и экспертность.

Преимущества: широкая линейка инструментов, наличие подробных описаний и обзоров, бонусная система (кешбэк) и удобный поиск по брендам. На главной странице четко расписаны преимущества и условия доставки.

Недостатки: дизайн сайта классический и довольно перегруженный текстом. Огромное количество баннеров, акций и текстовых блоков рассеивает внимание пользователя. С точки зрения UX/UI — это стандартная «лендинговая» верстка, которая не создает уникальной атмосферы и не передает атмосферу магазина. Из-за большого количества информации и скриптов сайт может загружаться медленнее.

1.2.3 Сайты «Pop-Music» и «МедиаМаркт»Опираясь на общие стандарты ниши, данные магазины также используют классическую схему: каталог с фильтрами, корзина, акцент на экологичность и пользу продукта.

Недостатки: как правило, такие сайты используют шаблонные решения (часто на базе готовых CMS) с базовой сеткой товаров. Они не предлагают пользователю интерактивного опыта и не выделяются визуально среди десятков аналогичных магазинов.

Выводы по результатам анализа:Проведенный анализ показал, что текущий рынок интернет-магазинов музыкальных инструментов перенасыщен классическими, тексто-ориентированными лендингами и каталогами с шаблонным дизайном. Основной упор конкуренты делают на ассортимент и скидки.

Наше конкурентное преимущество:
Разрабатываемый проект решает главную проблему конкурентов — отсутствие визуальной уникальности и современного интерактивного опыта (UX).
В отличие от проанализированных аналогов, наш интернет-магазин делает ставку на:

Иммерсивный дизайн: использование тёмной темы, качественной визуализации товаров и плавных анимаций, интерактивных карточек.

Современные технологии: высокая скорость работы благодаря отказу от тяжёлых CMS в пользу собственного легковесного решения на HTML, CSS и JavaScript с локальным хранением данных в JSON.

Чистый пользовательский опыт: отсутствие визуального шума и интуитивно понятный путь от просмотра каталога до быстрого оформления заказа.

1.3 Выбор технологического стека

1.3.1 Отказ от классических CMS в пользу собственной разработки

Для реализации проекта было принято решение отказаться от тяжеловесных систем управления контентом (CMS) вроде WordPress или Tilda в пользу собственной разработки (Custom Development). Это обусловлено необходимостью обеспечения высокой скорости загрузки и создания уникального интерфейса, который сложно реализовать на базе стандартных шаблонов.

1.3.2 Клиентская часть

Был выбран следующий технологический стек:

1.3.2.1 Клиентская часть

Современный CSS (Flexbox, Grid, clip-path, анимации) — используется для создания сложной геометрии интерфейса, адаптивной сетки каталога и плавных эффектов при наведении.

JavaScript (Vanilla JS) — обеспечивает работу клиентской логики: управление корзиной с сохранением данных в localStorage (что предотвращает потерю товаров при перезагрузке страницы), отрисовку динамических элементов из данных JSON через fetch API.

Для создания интерактивных 3D-карточек товаров, наклоняющихся за курсором, используются CSS-трансформации, transition и JavaScript-обработчики движения мыши.

1.3.3 Модель данных

1.3.3.1 Серверная часть (Backend):

HTML5 — семантическая разметка страниц и поддержка современных тегов и атрибутов форм.

JavaScript (Vanilla JS) — обеспечивает работу клиентской логики: корзины, фильтрации, валидации форм и отрисовки товаров.

клиентская логика (JavaScript Data Objects) — используется для подключения к базе данных. Применение подготовленных запросов (prepared statements) через клиентская логика обеспечивает высокий уровень безопасности, защищая веб-приложение от SQL-инъекций. Для размещения проекта был выбран платный статический хостинг GitHub Pages.

1.4 Проектирование структуры данных

1.4.1 Информационная модель системы

Для хранения информации о товарах и заказах в проекте используется локальный источник данных в формате JSON. Модель данных спроектирована с учётом требований целостности и связности информации.

Модель включает в себя следующие основные сущности:

- categories (Категории) — содержит информацию о категориях товаров (Гитары, Клавишные, Ударные, Оборудование), поля: id, name;

- products (Товары) — хранит информацию о продуктах. Поля: id, name (название), brand (бренд), description (описание), price (цена), discount (скидка), category (категория), rating (рейтинг), stock (наличие), image (путь к изображению).

- orders (Заказы) — содержит данные об оформленных заказах. Поля: id, customer (имя клиента), phone (телефон), address (адрес доставки), total (итоговая сумма), status (статус), createdAt (дата оформления).

- order_items (Позиции заказа) — связывает заказ и товары (корзину). Поля: id, productId (идентификатор товара), name (название товара на момент покупки), price (цена), quantity (количество).

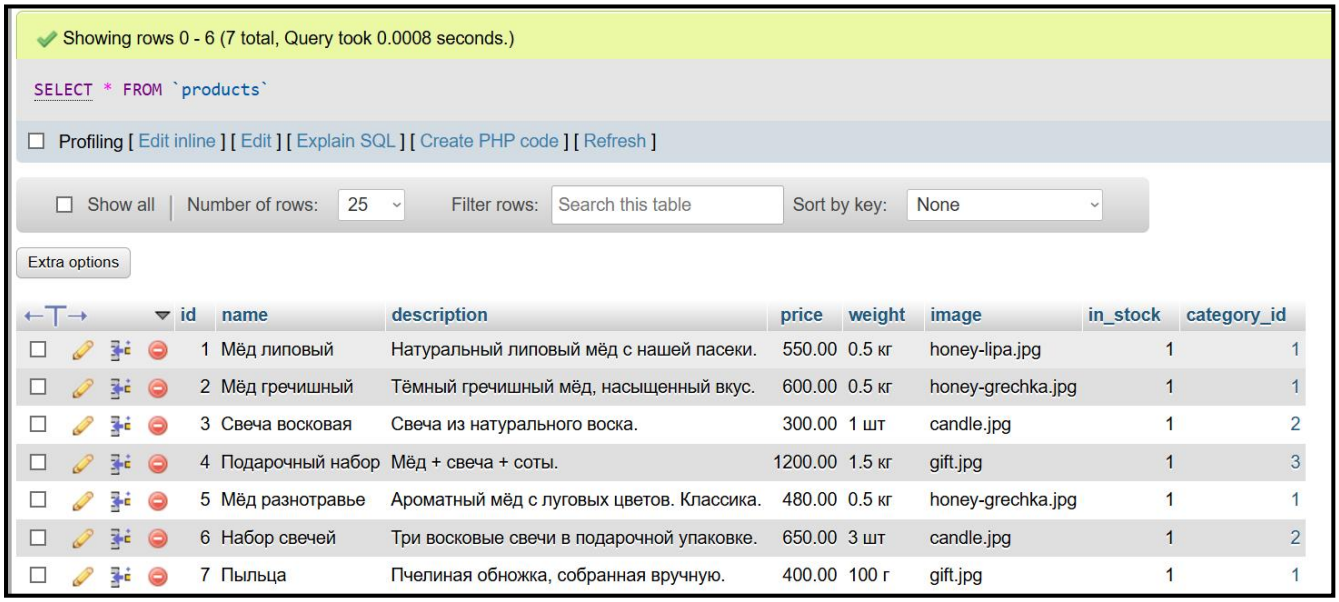
1.4.2 Связи между сущностями

Одна категория может содержать множество товаров (связь «один-ко-многим»).

Один заказ может содержать множество позиций (связь «один-ко-многим»).

Каждая позиция заказа привязана к конкретному товару.

Связи сущностей показаны на рисунке 1.



Showing rows 0 - 6 (7 total, Query took 0.0008 seconds.)

```
SELECT * FROM `products`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

id	name	description	price	weight	image	in_stock	category_id
1	Мёд липовый	Натуральный липовый мёд с нашей пасеки.	550.00	0.5 кг	honey-lipa.jpg	1	1
2	Мёд гречишный	Тёмный гречишный мёд, насыщенный вкус.	600.00	0.5 кг	honey-grechka.jpg	1	1
3	Свеча восковая	Свеча из натурального воска.	300.00	1 шт	candle.jpg	1	2
4	Подарочный набор	Мёд + свеча + соты.	1200.00	1.5 кг	gift.jpg	1	3
5	Мёд разнотравье	Ароматный мёд с луговых цветов. Классика.	480.00	0.5 кг	honey-grechka.jpg	1	1
6	Набор свечей	Три восковые свечи в подарочной упаковке.	650.00	3 шт	candle.jpg	1	2
7	Пыльца	Пчелиная обножка, собранная вручную.	400.00	100 г	gift.jpg	1	1

Рисунок 1 – Связи между сущностями

1.5 Выводы по первой главе

В первой главе были рассмотрены теоретические основы разработки интернет-магазина для локального бренда. В ходе анализа предметной области было выявлено, что музыкальные магазины нуждаются в прямых каналах сбыта, однако стандартные шаблонные решения (CMS, конструкторы сайтов) не позволяют создать уникальный и запоминающийся пользовательский опыт.

Был проведён сравнительный анализ популярных интернет-магазинов музыкальных инструментов («Музторг», «Pop-Music», «МедиаМаркт»). Анализ показал, что большинство конкурентов используют классические лендинги с обилием текста и стандартной сеткой товаров. Их главный недостаток — отсутствие визуальной уникальности и интерактивности, что особенно важно для привлечения современной аудитории.

На основе выявленных недостатков аналогов было сформулировано главное конкурентное преимущество разрабатываемого проекта — иммерсивный дизайн

(погружение в атмосферу интерфейса) и высокая скорость работы. Для реализации этой концепции было принято решение отказаться от тяжелых CMS в пользу собственной разработки (Custom Development).

Был обоснован выбор технологического стека: классические веб-технологии (HTML5, CSS3, Vanilla JS) для обеспечения высокой скорости загрузки и интерактивной визуализации, а также локальная модель данных на основе JSON для хранения каталога и заказов.

Кроме того, была спроектирована логическая структура данных, состоящая из четырёх взаимосвязанных сущностей (categories, products, orders, order_items), что обеспечивает целостность и связность данных при оформлении заказов.

Таким образом, теоретический анализ подтвердил актуальность и целесообразность разработки собственного клиентского приложения, которое выгодно выделит магазин среди конкурентов за счет уникального дизайна и современного подхода к взаимодействию с пользователем. Это создает прочную основу для перехода к практической реализации проекта.

2 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ИНТЕРНЕТ-МАГАЗИНА МУЗЫКАЛЬНЫХ ИНСТРУМЕНТОВ

2.1 Подготовка к разработке

2.1.1 Выбор хостинга

Для размещения и тестирования проекта был выбран статический хостинг GitHub Pages. Данный выбор обоснован следующими преимуществами:

1) поддержка актуальной версии JavaScript (8.2) и баз данных JSON-данные;

2) автоматическое продление SSL-сертификата для работы с базой данных;

3) возможность подключения по SSH для удобного деплоя проекта (загрузки файлов) через Git;

4) стабильность работы и оптимальная стоимость тарифа.

Для проекта был выделен технический поддомен soundcraft.github.io. В дальнейшем проект легко масштабируется на полноценный домен второго уровня.

2.1.2 Создание и настройка базы данных

Хранение информации о товарах и заказах реализовано с использованием реляционной СУБД JSON-данные. В панели управления GitHub Pages была создана новая база данных и пользователь с правами доступа к ней.

Для создания структуры таблиц был написан SQL-дамп (скрипт). В ходе его выполнения были созданы четыре таблицы: categories, products, orders и order_items (структура описана в п. 1.4).Также на этом этапе база данных была заполнена тестовым контентом: добавлены категории (Гитары, Клавишные, Ударные, Оборудование) и 22 вида товаров с ценами, описаниями и изображениями.

Созданная база данных продемонстрирована на рисунке 2.



Рисунок 2 – База данных

2.1.3 Настройка окружения и структура проекта

Серверная логика (Backend) реализована на языке JavaScript. В качестве основного инструмента для подключения к базе данных выбран объектно-ориентированный интерфейс клиентская логика (JavaScript Data Objects) с использованием подготовленных запросов (prepared statements). Это гарантирует защиту от SQL-инъекций и обеспечивает безопасность данных.Все конфиденциальные данные (логин, пароль и имя базы данных) были вынесены в отдельный конфигурационный файл .env, который, в целях безопасности, недоступен из браузера напрямую (расположен вне корневой папки сайта public_html).

Для удобства разработки была организована четкая файловая структура:

- app/ — серверная логика (классы, обработчики API, настройки подключения).
- public_html/ — корневая директория сайта (содержит доступные из браузера страницы, JS, CSS и изображения).
- public_html/api/ — эндпоинты для обработки запросов от клиента (получение товаров, отправка заказа).
- public_html/js/ — клиентские скрипты (корзина, главную страницу).
- img/ — изображения товаров и элементы оформления.

Данная структура проекта продемонстрирована на рисунке 3.

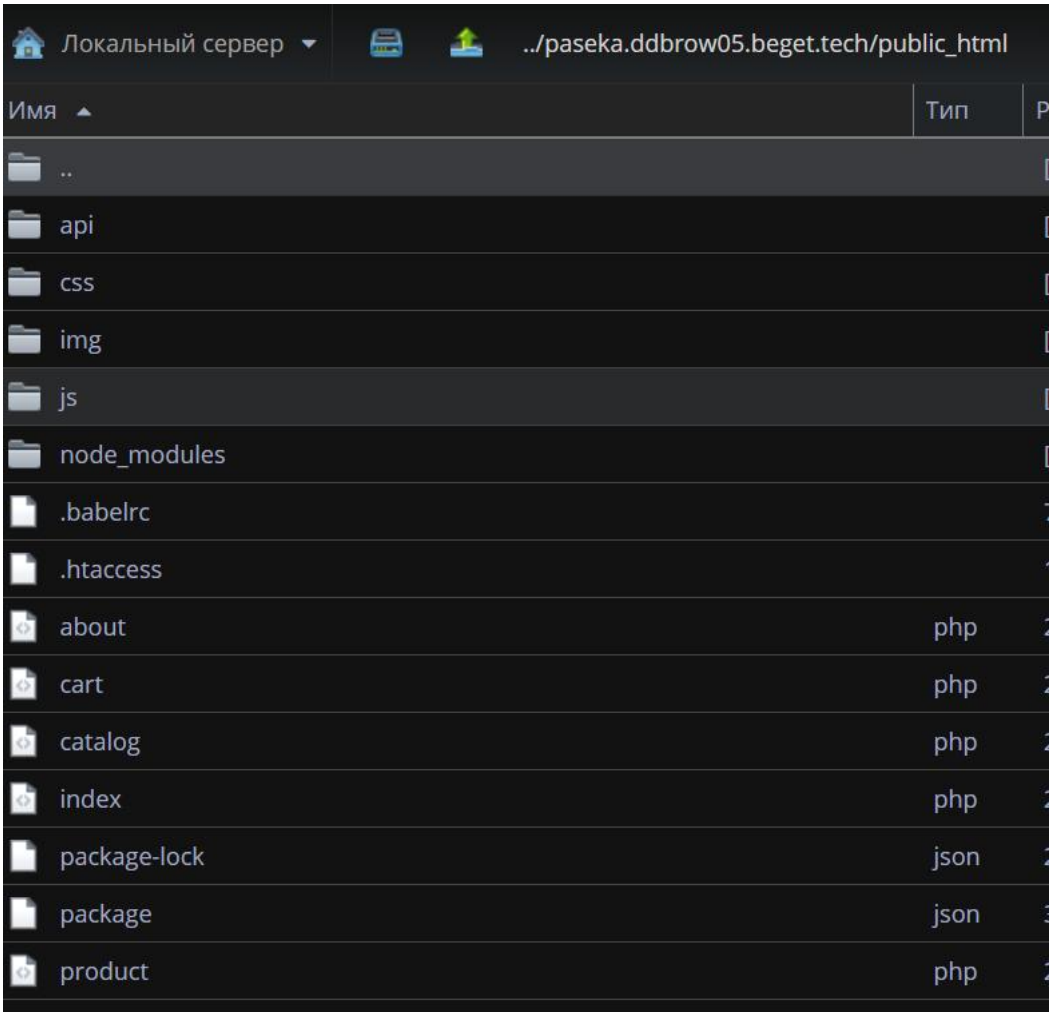


Рисунок 3 – Структура файлов

2.2 Реализация главной страницы и главную страницу

2.2.1 Главную страницу и логика его отображения

Главная страница открывает сайт и формирует первое впечатление пользователя. Для создания иммерсивного эффекта применена тёмная графитовая тема с изумрудным акцентом и качественной типографикой.

1) Обеспечение автовоспроизведения: современные браузеры (Chrome, Safari, Firefox) блокируют автоматическое воспроизведение видео со звуком. Для обхода этого ограничения в HTML-тег <video> были добавлены атрибуты muted (без звука) и playsinline (воспроизведение внутри контейнера на мобильных

устройствах iOS). Также был указан атрибут poster с изображением первого кадра, который отображается до момента загрузки видеопотока;

2) Создание интерфейса управления: Поверх видео с помощью абсолютного позиционирования были размещены элементы управления: кнопка «Пропустить» (для перехода к контенту без ожидания) и чекбокс «Больше не показывать»;

3) Логика работы на JavaScript: Скрипт hive.js отслеживает события жизненного цикла видео;

4) При нажатии на кнопку «Пропустить» или при событии ended (видео закончилось) скрипт добавляет классы к контейнеру, плавно скрывая видео (через изменение opacity и display: none) и показывая основной интерфейс сайта;

5) Для предотвращения раздражения пользователей реализована система кэширования: если чекбокс «Больше не показывать» активен, в localStorage записывается флаг (hive_intro_disabled = 1). При повторной загрузке страницы скрипт считывает этот флаг и пропускает этап с видео, сразу отображая каталог;

6) Интерфейс адаптирован под разные разрешения: на мобильных устройствах шапка сворачивается в бургер-меню, а элементы перестраиваются в одну колонку.

2.2.2 Создание меню навигации в виде карточек

После завершения видео (или его пропуска) открывается фон с изображением интерфейса, поверх которого размещено меню выбора категорий.Для создания уникального дизайна вместо стандартных кнопок или сетки были использованы шестиугольники (карточки).

2.2.2.1 Визуализация

Карточки выстроены в шахматном порядке с помощью абсолютного позиционирования и CSS-переменных, что продемонстрировано на рисунке 5.

Карточки категорий имеют аккуратные тени и плавные переходы цвета, что придаёт интерфейсу современный вид.

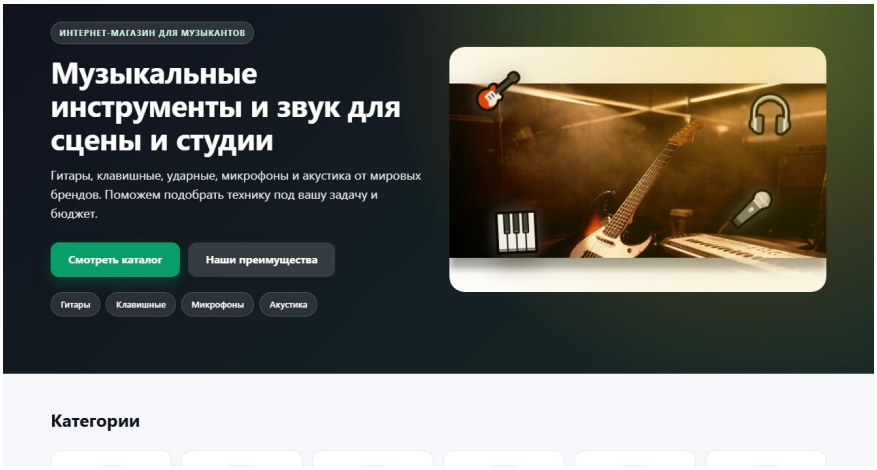


Рисунок 5 – Категории на главной странице

2.2.2.2 Интерактивность

При наведении курсора на карточку она плавно увеличивается (используется transform: scale и transition), а также меняет свечение, давая пользователю понять, что элемент кликабельный, что продемонстрировано на рисунке 6.



Рисунок 6 – Подсвеченная категория

2.2.2.3 Адаптивность

Размеры карточек задаются через CSS-функцию clamp(), которая динамически пересчитывает ширину и выкарточку элемента в зависимости от размера экрана устройства (от смартфона до ПК), благодаря чему меню всегда выглядит гармонично.

2.2.4 Проверка вёрстки на мобильных устройствах

Поскольку значительная часть пользователей заходит на сайты с мобильных устройств, интерфейс главной страницы был спроектирован с использованием подхода mobile-first. Это означает, что базовые стили написаны для мобильных экранов, а затем расширяются для планшетов и десктопов с помощью медиазапросов.

Учтены особенности «челок» и вырезов современных смартфонов: элементы интерфейса позиционируются с учётом отступов от краёв экрана. Проверена работоспособность сайта на мобильных браузерах, в том числе на iOS, где применяется специальная обработка области viewport.

2.2.3.1 Решение проблемы

Для надёжного отображения контента было применено комбинированное решение:

1) Для блока с контентом (.hive-content) был задан верхний отступ (padding-top), который на мобильных устройствах (через @media (max-width: 768px)) увеличивается. Это гарантирует, что заголовок и карточки не будут перекрыты шапкой сайта;

2) Размеры шрифтов и кнопок заданы через относительные единицы и clamp(), что корректно масштабируется от смартфона до ПК.

Особое внимание уделялось корректному отображению сетки каталога и корзины на узких экранах.

Для обхода этого ограничения был использован атрибут playsinline (запрещает видео открываться на весь экран в нативном плеере), а также предусмотрен резервный сценарий: если автовоспроизведение заблокировано, пользователю показывается статичная картинка (постер) и кнопка «Войти», нажатие на которую запускает видео или сразу переводит в каталог.

Вид мобильной версии проиллюстрирован на рисунке 4.



Рисунок 4 – Мобильная версия главной страницы

2.3. Разработка страницы каталога и визуализация товаров

2.3.1 Получение данных и серверная логика

Страница каталога (catalog.html) является ключевым элементом интернет-магазина, где пользователь выбирает товары. Для поддержания единой визуальной концепции сайта было принято решение отказаться от стандартной прямоугольной сетки карточек в пользу формы карточек товаров, продемонстрированные на рисунке 8.

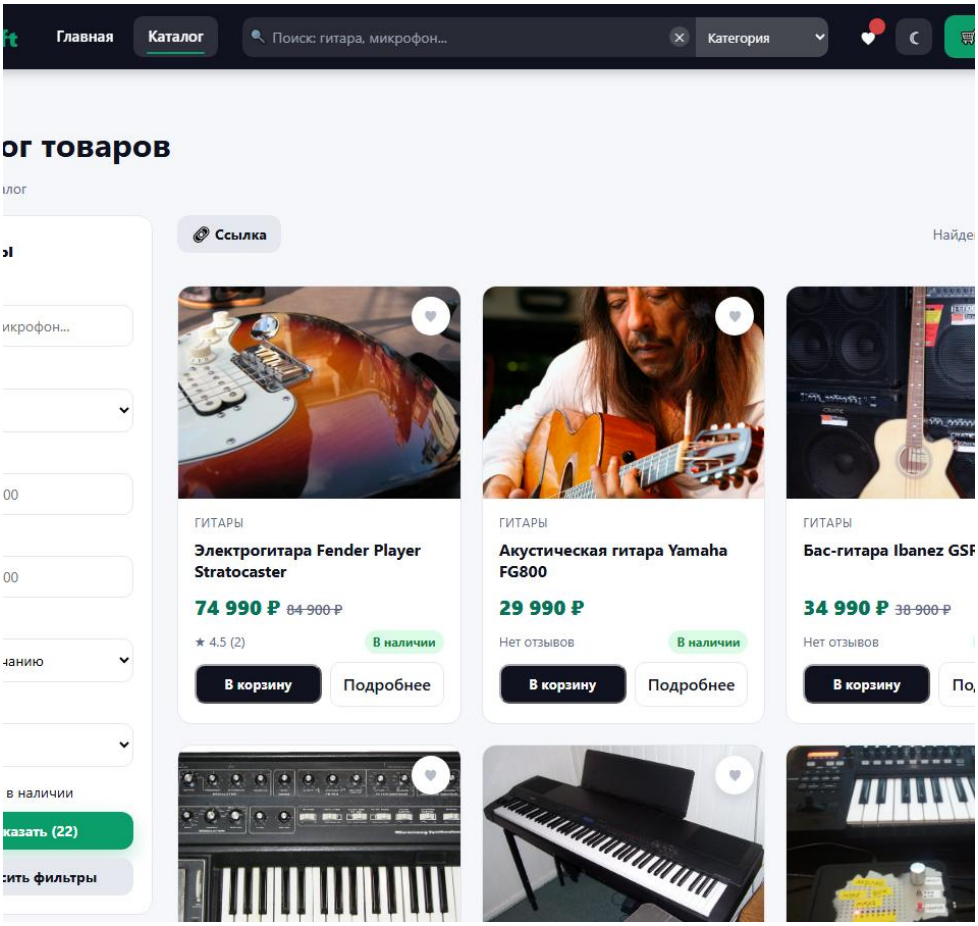


Рисунок 5 – Страница «Каталог»

Для удобного поиска реализованы: поиск по названию и бренду, фильтры по категории, диапазону цены, рейтингу и наличию, сортировка (по цене, новизне, рейтингу) и ленивая подгрузка товаров. Активные фильтры выводятся в виде тегов, которые можно быстро снять.

2.3.2 Верстка каталога в виде карточек

Сетка каталога построена на современном CSS: Flexbox и Grid задают ровные карточки, а ленивая загрузка изображений улучшает производительность;

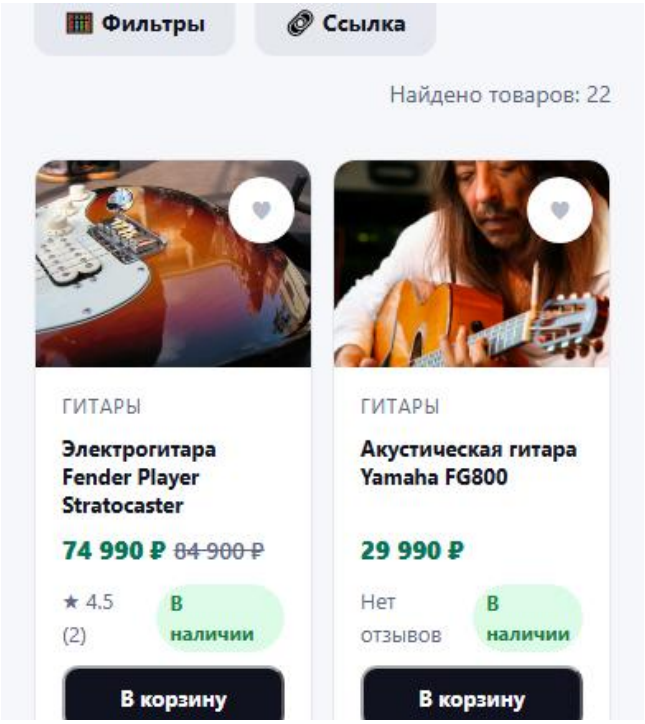
1) Каждая карточка товара содержит изображение, название, бренд, цену со скидкой, рейтинг, наличие и кнопку «В корзину»; при наведении карточка плавно приподнимается.

2) Изображения на карточках масштабируются без искажения пропорций и лениво подгружаются по мере прокрутки, что ускоряет загрузку каталога.

3) На мобильных устройствах (через @media) каталог выстраивается в две колонки, а панель фильтров сворачивается, чтобы не перегружать экран.

4) На мобильных устройствах (через @media) наложение рядов отключается, и карточки выстраиваются в простую вертикальную сетку (по 1 или 2 элемента в ряд), чтобы не перекрывать контент и не вылезать за пределы экрана.

Адаптированный вариант под мобильные устройства показан на рисунке 9.



2.4 Реализация страницы отдельного товара

2.4.1 Динамическое получение данных и защита от ошибок

Страница отдельного товара является ключевой точкой принятия решения о покупке. Задача данного этапа — предоставить пользователю подробную информацию о товаре и обеспечить удобное добавление в корзину, сохранив уникальный стиль проекта.

Страница является динамической: идентификатор товара передаётся через URL-параметр (например, product.html?id=5), по которому скрипт находит нужный товар и заполняет содержимое страницы.

Реализована обработка ошибок: если товара с таким ID нет, вместо ошибки отображается страница 404 со ссылкой для возврата в каталог.

Страница товара продемонстрирована на рисунке 7.

2.4.2 Оформление страницы товара

Визуально страница товара отличается аккуратной карточкой с фотографией, кнопками для выбора количества и добавления в корзину и в избранное. Галерея поддерживает миниатюры для переключения изображений без перезагрузки.

Информация о товаре (название, бренд, цена со скидкой, рейтинг, наличие, описание) выводится в отдельном блоке; рейтинг представлен звёздами и отзывами покупателей.

1) К контейнеру страницы (.product-page) применено свойство background-image с настройками background-size: cover и background-position: center. Это позволяет изображению карточек масштабироваться под любые размеры экрана без искажения пропорций;

2) Чтобы карточка с фотографией товара и блок с описанием оказались ровно в «черном окне» интерфейса, используется Flexbox-выравнивание (display: flex; align-items: center; justify-content: center). Дополнительно блок вырывается из

общей сетки сайта (через width: 100vw), чтобы фон занимал всю ширину монитора;

3)



Рисунок 7 – Страница товара

2.4.3 Блок описания и добавление в корзину

Проверена работа кнопки «Добавить в корзину», корректность пересчёта количества и обновления счётчика в шапке сайта.

Кнопка «Добавить в корзину» реализована на JavaScript. При клике формируется объект товара (ID, имя, цена), который сохраняется в массив в localStorage браузера. Пользователю мгновенно выводится всплывающее уведомление (тост) об успешном добавлении, а счетчик товаров в шапке сайта обновляется без перезагрузки страницы

Данный блок показан на рисунке 10.

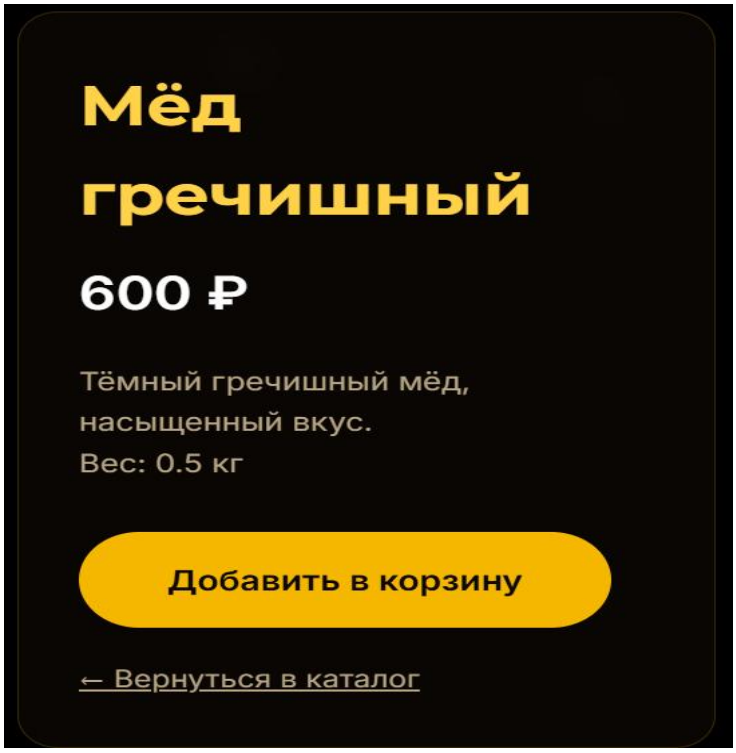


Рисунок 10 – Блок товара

2.5 Реализация корзины и оформления заказа

2.5.1 Клиентская логика работы корзины (localStorage)

Для предотвращения потери данных при перезагрузке страницы корзина хранится в клиентском хранилище localStorage, а не в серверных сессиях.

2.5.1.1 Структура данных

Корзина представляет собой массив объектов в формате JSON. Каждый объект содержит id товара, его name (название), price (цену) и quantity (количество).

2.5.1.2 Добавление товара

При клике на кнопку «В корзину» функция addToCart проверяет, есть ли товар в массиве: если есть — количество увеличивается на 1, если нет — добавляется новый объект. Обновлённый массив сериализуется JSON.stringify() и сохраняется в localStorage.

2.5.1.3 Изменение количества и удаление

На странице cart.html реализован динамический рендеринг таблицы. Пользователь может изменять количество товаров с помощью кнопок «+» и «−» или вводя число вручную. Скрипт перехватывает эти события, обновляет массив и мгновенно пересчитывает общую сумму (total), избавляя от необходимости перезагружать страницу, показанная на рисунке 11.

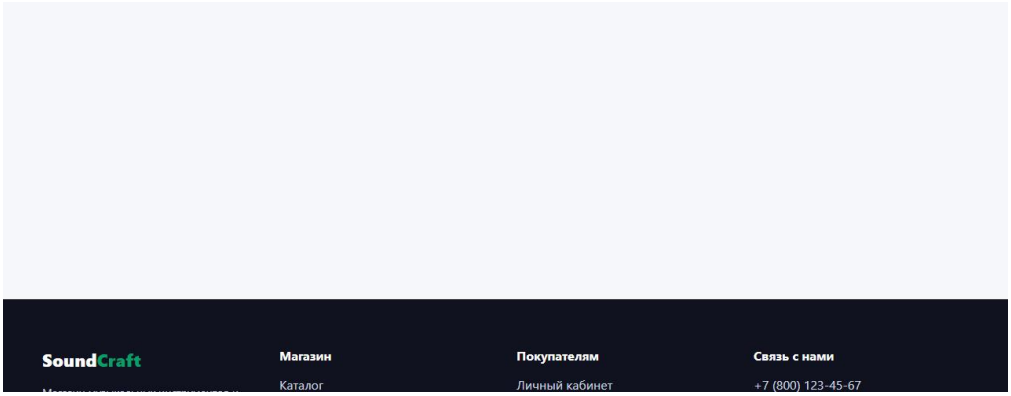


Рисунок 8 – Корзина

2.5.1.4 Синхронизация счетчика

Реализована поддержка промокодов (SOUND10, MUSIC20), уменьшающих итоговую сумму заказа. При любом изменении корзины функция updateCartCount() обновляет счётчик в шапке сайта.

2.5.2 Валидация формы заказа

Перед оформлением заказа заполненная форма проходит строгую проверку, чтобы исключить некорректные данные.

2.5.2.1 Валидация средствами html5

Поля «имя», «телефон» и «адрес» имеют атрибут required, который не позволяет отправить пустую форму. Для поля телефона задан атрибут pattern с регулярным выражением, а также атрибут title с подсказкой. Благодаря этому браузер самостоятельно блокирует отправку, если формат неверный, показанный на рисунке 12.

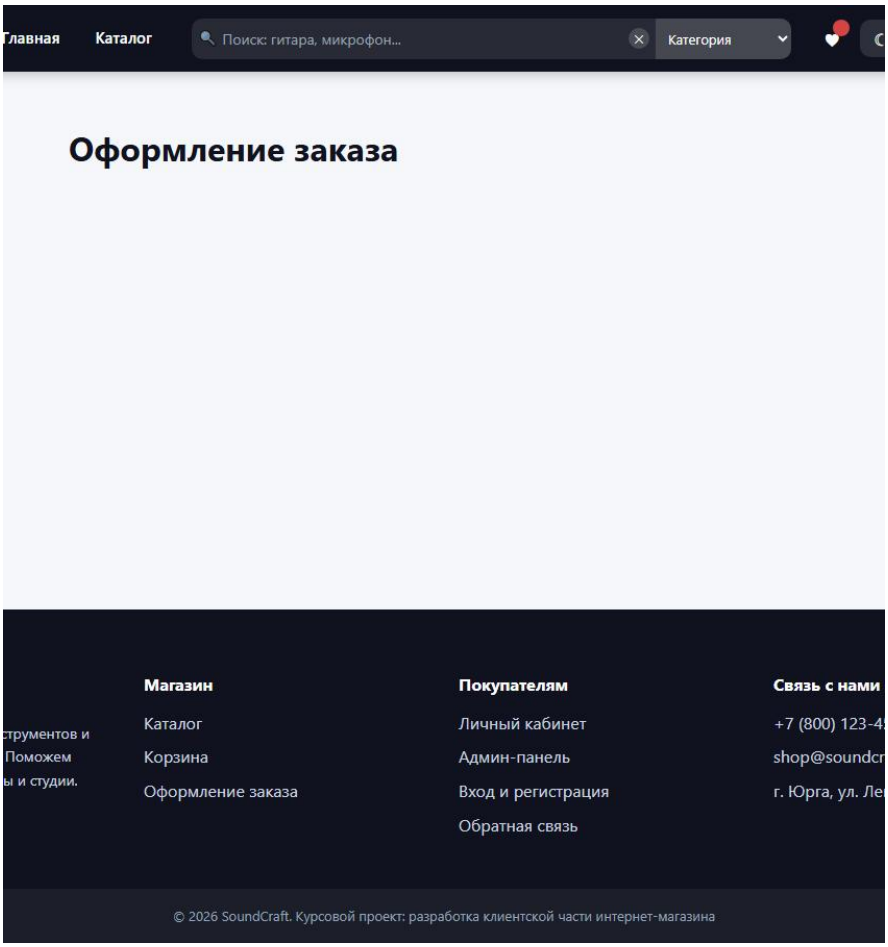


Рисунок 9 – Валидация оформления заказа

2.5.2.2 Валидация средствами javascript

Для обхода возможного отключения html5-валидации в браузере пользователя, в коде main.js реализована дополнительная проверка. Строка номера телефона проверяется регулярным выражением /^[\d\s\+\-\(\)]{10,20}$/.

Если проверка не пройдена, выводится красное уведомление и отправка блокируется.

2.5.3 Асинхронная отправка данных (Fetch API)

Процесс оформления заказа реализован без перезагрузки страницы:

1) При нажатии на кнопку «Оформить заказ» срабатывает обработчик события submit, который отменяет стандартное поведение браузера (e.preventDefault());

2) Кнопка отправки блокируется (submitBtn.disabled = true) и меняет текст на «Отправка...», чтобы предотвратить случайное дублирование заказов;

3) Данные формы и содержимое корзины объединяются в единый объект и отправляются методом POST на серверный эндпоинт /api/order.html с заголовком Content-Type: application/json;

4) после успешного оформления корзина очищается (clearCart()), форма сбрасывается, а пользователь видит сообщение об успешном создании заказа.

2.6 Подготовка краткого руководства оператора

2.6.1 Руководство для клиента

1. Просмотр товаров: откройте главную страницу сайта, выберите нужную категорию или используйте поиск в шапке сайта.
2. Добавление в корзину: в каталоге нажмите на товар, чтобы открыть детальную страницу, затем нажмите «Добавить в корзину».
3. Оформление заказа: перейдите в «Корзину», отрегулируйте количество товаров, примените промокод при необходимости, заполните форму (имя, телефон в формате +7..., адрес) и нажмите «Оформить заказ».
4. Отслеживание заказа: после оформления вы можете зарегистрироваться, чтобы в личном кабинете видеть историю покупок и статусы заказов. Пример личного кабинета показан на рисунке 10.

2.6.2 Руководство для администратора

1. Вход в систему: нажмите на значок пользователя и выберите «Войти», введите служебный email (admin@ya.ru) и пароль; система перенаправит вас в административную панель. Вид админ-панели показан на рисунке 11.
2. Управление заказами: в админ-панели отображается таблица заказов с контактными данными и суммами. Измените статус (Новый, Подтверждён, Собран, Отправлен, Доставлен, Отменён) — клиент получит уведомление; статус обновится в личном кабинете пользователя.
3. Управление товарами: в разделе «Товары» доступно добавление нового товара (название, категория, цена, скидка, изображения), редактирование, дублирование и удаление. Обратная связь обрабатывается в разделе «Сообщения».

2.8 Выводы по второй главе

Во второй главе была выполнена практическая реализация интернет-магазина:

Подготовительный этап: выбран статический хостинг GitHub Pages, настроена файловая структура проекта (разделение на HTML, CSS, JS, данные и изображения).

Разработка интерфейса: создана главная страница с тёмной темой и иммерсивными элементами, реализован каталог с поиском, фильтрами, сортировкой и ленивой подгрузкой.

Клиентская логика: реализована корзина на основе localStorage с промокодами, настроена валидация форм, разработана система авторизации и личный кабинет.

Данные и админка: реализована локальная модель данных на основе JSON и административная панель для управления товарами, заказами и сообщениями с уведомлениями клиента.

Тестирование: проверены все ключевые сценарии, адаптивность и безопасность ввода данных.

В результате создан полностью функционирующий, адаптивный интернет-магазин, готовый к внедрению.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был успешно разработан полноценный интернет-магазин музыкальных инструментов. Все задачи, поставленные во введении, были выполнены в полном объеме.

В теоретической части был проведен анализ предметной области, выявлены потребности локальных производителей в прямых каналах сбыта. Сравнительный анализ конкурентов («Музторг», «Pop-Music», «МедиаМаркт») показал, что большинство существующих сайтов используют шаблонный дизайн. На основе этого было обосновано решение отказаться от готовых CMS в пользу собственной разработки для создания уникального пользовательского опыта (UX) и высокой скорости загрузки.

В практической части была реализована как клиентская, так и серверная часть приложения:

Дизайн и интерфейс: разработан уникальный визуальный стиль с использованием тёмной темы, качественной визуализации товаров и плавных анимаций.

Клиентская логика: свёрстан каталог с поиском и фильтрами, реализована адаптивная корзина на базе localStorage, настроена строгая валидация формы заказа.

Данные: создана локальная модель хранения каталога и заказов на основе JSON, обеспечивающая связность сущностей и простоту развёртывания.

Ролевая модель: разработана система авторизации, реализован личный кабинет с историей заказов и административная панель, позволяющая управлять статусами заказов без перезагрузки страницы.

Адаптивность: Интерфейс протестирован на различных устройствах и корректно масштабируется.

Таким образом, цель курсовой работы достигнута. Разработанный проект представляет собой готовый к внедрению интернет-магазин, который выделяется среди конкурентов за счет нестандартного, иммерсивного дизайна и современной технической реализации. В дальнейшем проект можно масштабировать, добавив модуль онлайн-оплаты и полноценную панель администратора для управления товарами.

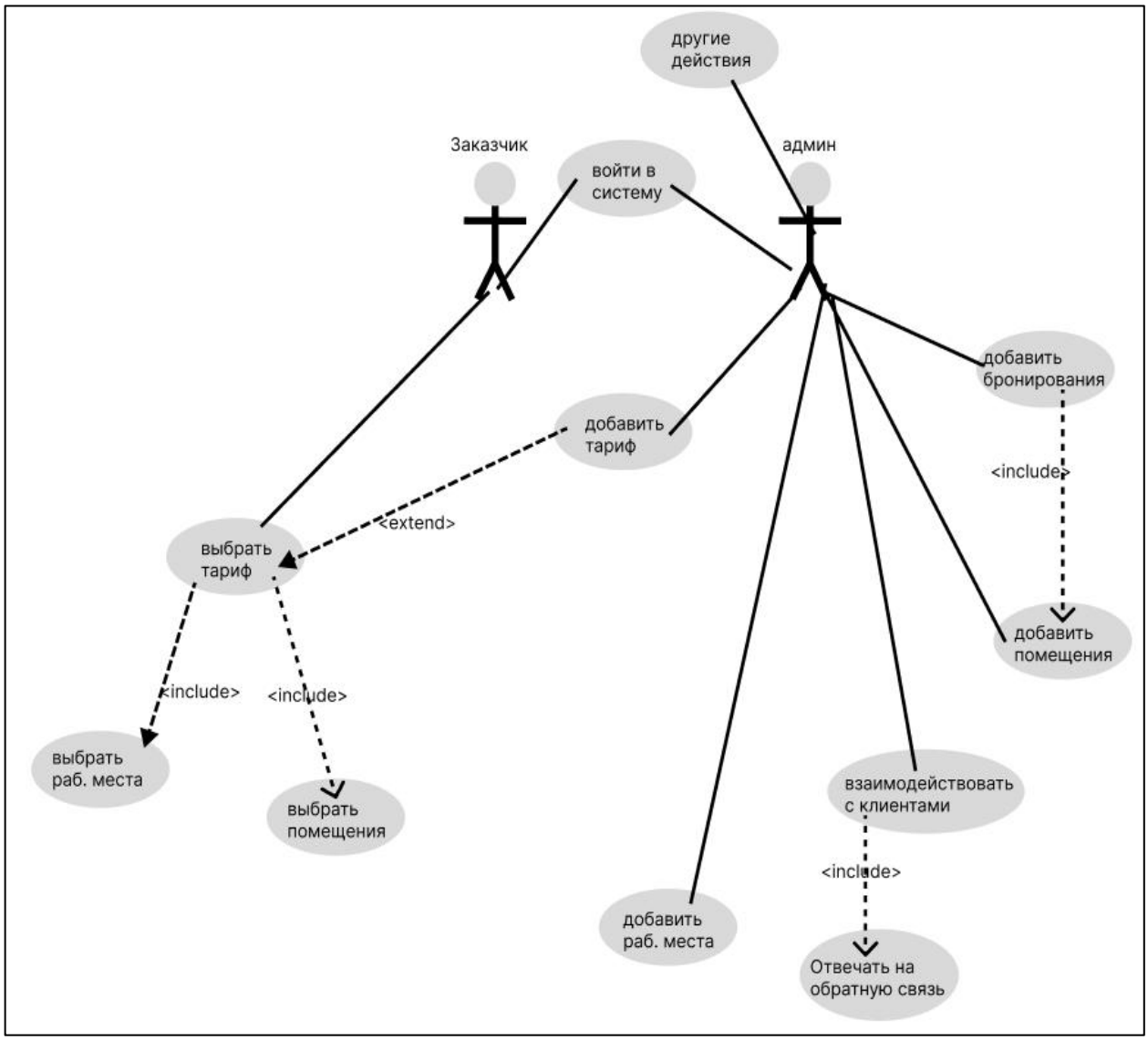
Таким образом, цель курсовой работы достигнута, разработанная система полностью соответствует требованиям технического задания и может быть рекомендована к внедрению.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. WordPress.org. Easy!Appointments – WordPress плагин для бронирования [Электронный ресурс]. — Режим доступа: https://wordpress.org/plugins/easyappointments/ (дата обращения: 17.03.2026).
2. WordPress.org. AdvancedCustomFields – WordPress плагин для произвольных полей [Электронный ресурс]. — Режим доступа: https://wordpress.org/plugins/advanced-custom-fields/ (дата обращения: 17.03.2026).
3. GitHub Pages. Хостинг-провайдер [Электронный ресурс]. — Режим доступа: https://github pages.com (дата обращения: 17.03.2026).
4. Easy!Appointments. Документация разработчика [Электронный ресурс]. — Режим доступа: https://easyappointments.org/docs (дата обращения: 17.03.2026).

ПРИЛОЖЕНИЕ А

Связи сущностей



ПРИЛОЖЕНИЕ Б

Оформление страниц

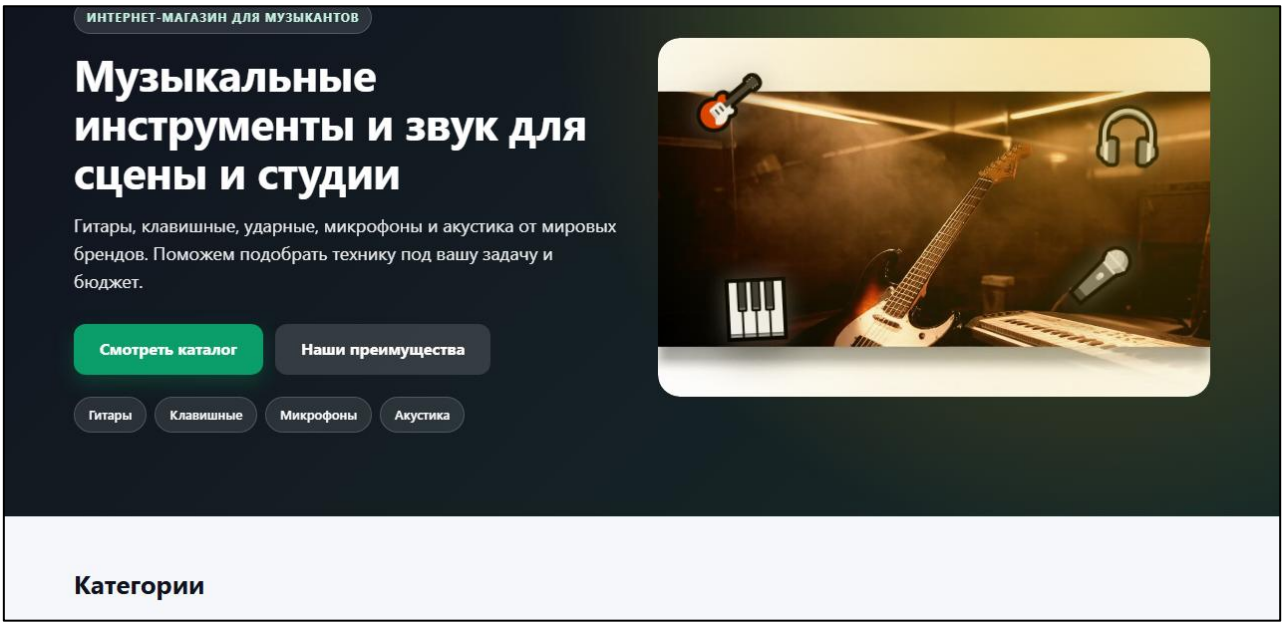


Рисунок А.1 - Схема связей между сущностями